

PHẦN CỐT LÕI gRPC

Từ cơ bản đến nâng cao với Golang

Tài liệu giảng dạy thực hành — Phần 2

Tác giả: Giảng viên Lập trình

Phiên bản: 1.0

Ngày: Tháng 4 năm 2025

Yêu cầu: Go 1.22+ | protoc v25.x | grpc v1.64+

Mục lục

Mục lục	2
Lời mở đầu	4
Mục tiêu tài liệu	4
Cách sử dụng hiệu quả	4
2.1 — Kiến trúc gRPC và các thành phần chính	5
Các thành phần cốt lõi	5
Vòng đời một gRPC request	5
Mở rộng project	5
2.2 — Cấu trúc file .proto	7
File .proto hoàn chỉnh với chú thích	7
Quy tắc đặt tên chuẩn Protobuf	8
Generate code	8
2.3 — Unary RPC	9
Triển khai Server	9
Triển khai Client	10
Chạy thử	11
2.4 — Server Streaming RPC	13
Server Implementation — ListUsers	13
Client Implementation — Nhận stream	13
2.5 — Client Streaming RPC	15
Server Implementation — BatchCreate	15
Client Implementation — Gửi stream	15
2.6 — Bidirectional Streaming RPC	17
Server Implementation — Chat	17
Client Implementation — Gửi và nhận đồng thời	17
2.7 — Generated Code trong Go — Hiểu để dùng đúng	19
user.pb.go — Message types	19
user_grpc.pb.go — Service interfaces	19
Makefile cập nhật	20
2.8 — Triển khai gRPC Server hoàn chỉnh	21
Server với cấu hình đầy đủ	21
Graceful Shutdown vs Stop	22
2.9 — Triển khai gRPC Client hoàn chỉnh	23
Connection Management	23
2.10 — Deadline và Timeout	25
Sự khác biệt: Timeout vs Deadline	25
Deadline propagation — tại sao dùng Deadline thay Timeout?	25
Ví dụ thực tế	25
2.11 — Context Propagation	27

Context mang những gì?	27
Truyền context trong service chain	27
2.12 — Metadata trong gRPC	29
Gửi Metadata từ Client	29
Đọc Metadata ở Server	29
Header vs Trailer	30
2.13 — Error Handling và Status Codes	32
Các Status Code quan trọng nhất	32
Tạo và trả lỗi đúng cách	32
Lỗi nào có thể Retry?	33
2.14 — Interceptors (Middleware trong gRPC)	35
Hai loại Interceptor	35
Logging Interceptor	35
Đăng ký Interceptors với Server	36
Client-side Interceptor	37
2.15 — Reflection và grpcurl	38
Bật gRPC Reflection	38
Cài grpcurl	38
Các lệnh grpcurl thường dùng	38
2.16 — TLS / mTLS	40
TLS vs mTLS	40
Tạo self-signed certificates (Development)	40
Server với TLS	40
Client với TLS	41
2.17 — Keepalive và Connection Management	43
Vấn đề: Kết nối "zombie"	43
Cấu hình Keepalive đầy đủ	43
Bảng tổng hợp các Server Parameters	44
2.18 — Tổng kết	45
Bảng ôn tập toàn tài liệu	45
Lộ trình tiếp theo	45
Tài nguyên học tập	45
Docs chính thức	46
Repositories quan trọng	46
Sách	46
Công cụ Production	46
Lỗi thường gặp — Tổng hợp	46

Lời mở đầu

Mục tiêu tài liệu

Đây là phần tiếp theo sau khi bạn đã nắm vững kiến thức nền (networking, HTTP/2, RPC, Protobuf, goroutine). Tài liệu này đưa bạn vào thực chiến với gRPC: từ viết server đầu tiên, triển khai 4 loại streaming, cho đến các kỹ thuật production như TLS, interceptor, metadata, keepalive.

Mỗi mục được thiết kế để bạn có thể mở rộng dần project đã tạo từ tài liệu trước — không phải bắt đầu lại từ đầu.

Cách sử dụng hiệu quả

- Mở project grpc-prereqs từ tài liệu trước và thêm code theo từng mục
- Chạy từng ví dụ ngay khi đọc — đừng chờ hết chương
- Dùng grpcurl (Mục 2.15) để test service sau mỗi bước triển khai
- Mục 2.18 là tổng kết — quay lại khi cần ôn nhanh



Cấu trúc project mở rộng

Tất cả code trong tài liệu này được thêm vào project grpc-prereqs đã tạo. Chúng ta sẽ thêm thư mục grpc-core/ chứa server/, client/ và các module con.

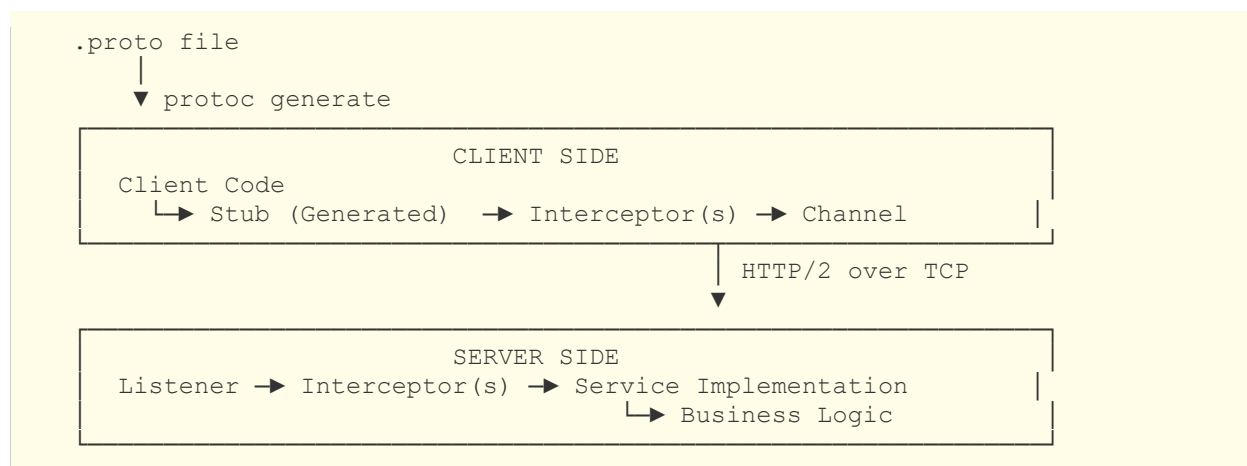
2.1 — Kiến trúc gRPC và các thành phần chính

Trước khi viết một dòng code gRPC, bạn cần hiểu bức tranh toàn cảnh: các thành phần nào tham gia và chúng giao tiếp với nhau thế nào.

Các thành phần cốt lõi

Thành phần	Vai trò	Tương đương trong Go
Service Definition (.proto)	Hợp đồng API — định nghĩa messages và methods	Interface + struct definitions
Stub (Generated Client)	Code client tự động được generate — gọi RPC như gọi hàm local	Generated *Client struct
Server Implementation	Code bạn viết để thực thi logic của service	Struct implement interface
Channel	Kết nối HTTP/2 giữa client và server, hỗ trợ nhiều streams	*grpc.ClientConn
Interceptor	Middleware — chặn request/response để logging, auth, metrics	UnaryInterceptor / StreamInterceptor

Vòng đời một gRPC request



Mở rộng project

Thêm thư mục `grpc-core` vào project `grpc-prereqs` đã tạo:

terminal
<code>cd grpc-prereqs</code>
<code>mkdir -p grpc-core/{proto,gen,server,client}</code>
<code>mkdir -p grpc-core/proto/user</code>

Tóm tắt

- gRPC = .proto (contract) + generated code (stub) + implementation (logic)
- Client dùng Stub để gọi RPC — không viết network code thủ công
- Interceptor là middleware — xử lý cross-cutting concerns như auth, logging

- Channel = kết nối HTTP/2 persistent, tái sử dụng cho nhiều request

Bài tập tự luyện

1. Vẽ lại sơ đồ vòng đời gRPC request theo cách hiểu của bạn, không nhìn tài liệu.
2. Interceptor trong gRPC giống với middleware trong HTTP framework nào bạn đã biết? So sánh.

2.2 — Cấu trúc file .proto

File .proto là trái tim của mọi gRPC service. Hãy xem xét kỹ cấu trúc và các quy tắc quan trọng.

File .proto hoàn chỉnh với chú thích

grpc-core/proto/user/user.proto
// ❶ Phiên bản Protobuf – luôn dùng proto3
syntax = "proto3";
// ❷ Package tránh xung đột tên khi nhiều .proto import nhau
package user;
// ❸ Import các well-known types từ Google
import "google/protobuf/timestamp.proto";
import "google/protobuf/empty.proto";
// ❹ Go package path – phải khớp với module path trong go.mod
option go_package = "grpc-prereqs/grpc-core/gen/user";
// — Messages —————
// User là entity chính
message User {
int32 id = 1;
string name = 2;
string email = 3;
Role role = 4;
// Timestamp từ well-known types – tránh viết lại
google.protobuf.Timestamp created_at = 5;
}
// Enum phân loại role
enum Role {
ROLE_UNSPECIFIED = 0; // Luôn có giá trị 0 trong proto3
ROLE_USER = 1;
ROLE_ADMIN = 2;
}
// Request/Response messages cho từng RPC
message GetUserRequest { int32 id = 1; }
message CreateUserRequest { string name = 1; string email = 2; }
message ListUsersRequest { int32 page = 1; int32 limit = 2; }
message DeleteUserRequest { int32 id = 1; }
// — Service —————
// UserService định nghĩa tất cả RPC methods
service UserService {
// Unary: 1 request → 1 response
rpc GetUser (GetUserRequest) returns (User);
rpc CreateUser (CreateUserRequest) returns (User);
rpc DeleteUser (DeleteUserRequest) returns (google.protobuf.Empty);
// Server streaming: 1 request → nhiều response
rpc ListUsers (ListUsersRequest) returns (stream User);
// Client streaming: nhiều request → 1 response
rpc BatchCreate (stream CreateUserRequest) returns (User);
// Bidirectional streaming: nhiều request ↔ nhiều response
rpc Chat (stream ChatMessage) returns (stream ChatMessage);
}

```
message ChatMessage { int32 from id = 1; string text = 2; }
```

Quy tắc đặt tên chuẩn Protobuf

Thứ gì	Quy tắc	Ví dụ
Message	PascalCase	GetUserRequest, UserList
Field	snake_case	created_at, first_name
Enum type	PascalCase	Role, UserStatus
Enum value	SCREAMING_SNAKE_CASE	ROLE_ADMIN, STATUS_ACTIVE
Service	PascalCase + 'Service'	UserService, OrderService
RPC method	PascalCase (động từ)	GetUser, CreateOrder

Generate code

terminal

```
# Từ thư mục grpc-prereqs
protoc \
  --proto_path=grpc-core/proto \
  --go_out=grpc-core/gen \
  --go_opt=paths=source_relative \
  --go-grpc_out=grpc-core/gen \
  --go-grpc_opt=paths=source_relative \
  grpc-core/proto/user/user.proto

# Kết quả:
# grpc-core/gen/user/user.pb.go      ← structs + serialization
# grpc-core/gen/user/user_grpc.pb.go ← server/client interfaces
```

Tóm tắt

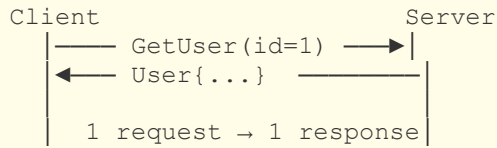
- syntax, package, option go_package là 3 khai báo đầu bắt buộc
- Import google/protobuf/timestamp.proto và empty.proto cho common types
- Đặt tên theo chuẩn: message PascalCase, field snake_case, enum SCREAMING_SNAKE
- Một file .proto nên mô tả một domain/entity — không nhét tất cả vào một file

Bài tập tự luyện

3. Viết file .proto cho OrderService với Order message (id, user_id, total, status) và 3 RPC: CreateOrder, GetOrder, ListOrders.
4. Tại sao import google/protobuf/timestamp.proto thay vì tự viết message Timestamp?

2.3 — Unary RPC

Unary RPC là kiểu đơn giản nhất: client gửi 1 request, server trả về 1 response. Giống hệt REST API nhưng qua gRPC.



Triển khai Server

grpc-core/server/main.go
package main
import (
"context"
"fmt"
"log"
"net"
"sync"
pb "grpc-prereqs/grpc-core/gen/user"
"google.golang.org/grpc"
"google.golang.org/grpc/codes"
"google.golang.org/grpc/status"
)
// userServer implement interface pb.UserServiceServer (generated)
type userServer struct {
pb.UnimplementedUserServiceServer // Nhúng để forward-compatible
mu sync.RWMutex
users map[int32]*pb.User
nextID int32
}
func newUserServer() *userServer {
return &userServer{
users: make(map[int32]*pb.User),
nextID: 1,
}
}
// GetUser - Unary RPC: tìm user theo ID
func (s *userServer) GetUser(ctx context.Context, req *pb.GetUserRequest) (*pb.User, error) {
s.mu.RLock()
defer s.mu.RUnlock()
user, ok := s.users[req.Id]
if !ok {
// Trả lỗi chuẩn gRPC - không dùng fmt.Errorf thông thường
return nil, status.Errorf(codes.NotFound, "user %d không tồn tại", req.Id)
}
return user, nil
}
// CreateUser - Unary RPC: tạo user mới
func (s *userServer) CreateUser(ctx context.Context, req *pb.CreateUserRequest) (*pb.User, error) {
// Validation cơ bản

if req.Name == "" {
return nil, status.Error(codes.InvalidArgument, "name không được để trống")
}
s.mu.Lock()
defer s.mu.Unlock()
user := &pb.User{
Id: s.nextID,
Name: req.Name,
Email: req.Email,
Role: pb.Role_ROLE_USER,
}
s.users[s.nextID] = user
s.nextID++
log.Printf("Đã tạo user: %v", user)
return user, nil
}
func main() {
// Lắng nghe kết nối TCP trên port 50051
lis, err := net.Listen("tcp", ":50051")
if err != nil {
log.Fatalf("Không thể listen: %v", err)
}
// Tạo gRPC server
grpcServer := grpc.NewServer()
// Đăng ký service implementation với server
pb.RegisterUserServiceServer(grpcServer, newUserServer())
fmt.Println("gRPC Server khởi động tại :50051")
// Bắt đầu serve - block ở đây
if err := grpcServer.Serve(lis); err != nil {
log.Fatalf("Lỗi serve: %v", err)
}
}

Triển khai Client

grpc-core/client/main.go
package main
import (
"context"
"fmt"
"log"
"time"
pb "grpc-prereqs/grpc-core/gen/user"
"google.golang.org/grpc"
"google.golang.org/grpc/credentials/insecure"
)
func main() {
// Dial: kết nối đến gRPC server
// grpc.WithTransportCredentials(insecure.NewCredentials()) = không dùng TLS
conn, err := grpc.NewClient(
"localhost:50051",
grpc.WithTransportCredentials(insecure.NewCredentials()),

)
if err != nil {
log.Fatalf("Không thể kết nối: %v", err)
}
defer conn.Close()
// Tạo client stub từ connection
client := pb.NewUserServiceClient(conn)
// Tạo context với timeout 5 giây
ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()
// — Gọi CreateUser (Unary RPC) —
createdUser, err := client.CreateUser(ctx, &pb.CreateUserRequest{
Name: "Nguyen Van An",
Email: "an@example.com",
})
if err != nil {
log.Fatalf("CreateUser thất bại: %v", err)
}
fmt.Printf("Đã tạo: %v\n", createdUser)
// — Gọi GetUser (Unary RPC) —
fetchedUser, err := client.GetUser(ctx, &pb.GetUserRequest{Id: createdUser.Id})
if err != nil {
log.Fatalf("GetUser thất bại: %v", err)
}
fmt.Printf("Lấy về: %v\n", fetchedUser)
}

Chạy thử

terminal
Terminal 1
go run grpc-core/server/main.go
gRPC Server khởi động tại :50051
Terminal 2
go run grpc-core/client/main.go
Đã tạo: id:1 name:"Nguyen Van An" email:"an@example.com"
Lấy về: id:1 name:"Nguyen Van An" email:"an@example.com"

⚠ pb.UnimplementedUserServiceServer

Luôn nhúng pb.UnimplementedUserServiceServer vào server struct. Nếu bạn thêm RPC mới vào .proto nhưng chưa implement, server vẫn compile được và tự trả lỗi UNIMPLEMENTED thay vì panic.

🔗 Tóm tắt

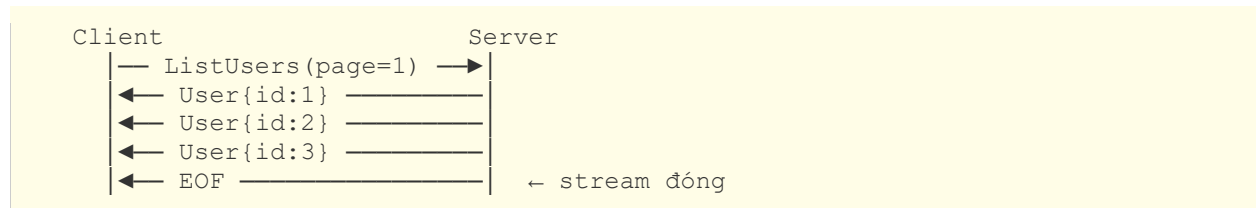
- Server implement interface generated từ .proto, đăng ký với grpc.NewServer()
- Client dùng grpc.NewClient() + NewUserServiceClient(conn) để tạo stub
- Dùng status.Errorf(codes.X, ...) thay vì fmt.Errorf() để trả lỗi chuẩn gRPC
- context.WithTimeout() đảm bảo request không bị treo vô hạn

 Bài tập tự luyện

5. Thêm RPC method `UpdateUser(UpdateUserRequest)` returns `(User)` vào server. Xử lý trường hợp user không tồn tại.
6. Viết client gọi `GetUser` với một ID không tồn tại và in ra mã lỗi (`codes.NotFound`).
7. Tại sao chúng ta dùng `sync.RWMutex` thay vì `sync.Mutex`? Khi nào dùng `RLock` vs `Lock`?

2.4 — Server Streaming RPC

Server Streaming: client gửi 1 request, server gửi ngược lại NHIỀU response theo thời gian.
Hữu ích cho: danh sách lớn, live data, progress, log tailing.



Server Implementation — ListUsers

grpc-core/server/main.go (thêm method)
// ListUsers - Server Streaming: gửi từng user về cho client
func (s *userService) ListUsers(req *pb.ListUsersRequest, stream pb.UserService_ListUsersServer) error {
s.mu.RLock()
defer s.mu.RUnlock()
sent := int32(0)
limit := req.Limit
if limit <= 0 {
limit = 10 // Giá trị mặc định
}
for _, user := range s.users {
// Kiểm tra context - client có thể cancel bất cứ lúc nào
if err := stream.Context().Err(); err != nil {
return status.Error(codes.Canceled, "client đã hủy")
}
// Gửi từng user về client - server push
if err := stream.Send(user); err != nil {
// Send() lỗi = kết nối với client bị đứt
return err
}
sent++
if sent >= limit {
break
}
}
// return nil = stream kết thúc bình thường (server gửi EOF)
return nil
}

Client Implementation — Nhận stream

grpc-core/client/streaming.go
package main
import (
"context"
"fmt"
"io"
"log"
"time"
)

```

pb "grpc-prereqs/grpc-core/gen/user"
"google.golang.org/grpc"
"google.golang.org/grpc/credentials/insecure"
)

func demoServerStream(client pb.UserServiceClient) {
    ctx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
    defer cancel()

    // Gọi ListUsers — trả về stream object, không phải kết quả ngay
    stream, err := client.ListUsers(ctx, &pb.ListUsersRequest{Limit: 5})
    if err != nil {
        log.Fatalf("Không thể gọi ListUsers: %v", err)
    }

    // Nhận từng user từ stream trong vòng lặp
    for {
        user, err := stream.Recv()
        if err == io.EOF {
            // io.EOF = server đã gửi hết và đóng stream
            fmt.Println("Stream kết thúc (EOF)")
            break
        }
        if err != nil {
            log.Printf("Lỗi nhận stream: %v", err)
            break
        }
        fmt.Printf("Nhận user: id=%d name=%s\n", user.Id, user.Name)
    }
}

```

Tóm tắt

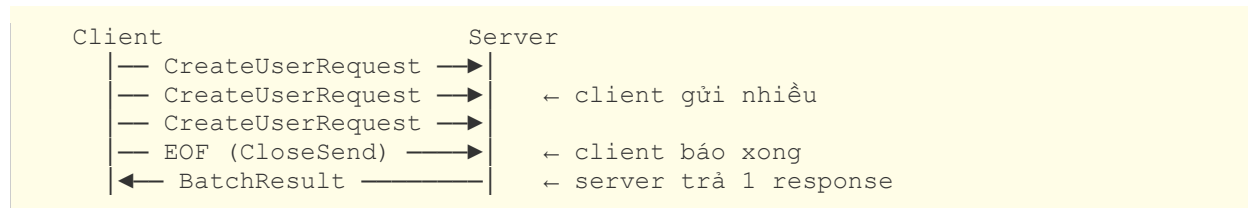
- Server dùng `stream.Send(msg)` để gửi từng message — có thể gọi nhiều lần
- Server return nil = stream kết thúc bình thường; client nhận `io.EOF`
- Client dùng vòng lặp + `stream.Recv()` cho đến khi gặp `io.EOF`
- Luôn kiểm tra `stream.Context().Err()` để phát hiện client cancel sớm

Bài tập tự luyện

- Thêm RPC `WatchUsers(WatchRequest)` returns (stream `UserEvent`) để server push event khi có user mới tạo.
- Viết server streaming trả về số từ 1-100 với delay 100ms giữa mỗi số. Client in ra và tính tổng.
- Điều gì xảy ra nếu server gọi `stream.Send()` sau khi context đã hết hạn?

2.5 — Client Streaming RPC

Client Streaming: client gửi NHIỀU request theo thời gian, server nhận hết rồi trả về 1 response. Phù hợp: upload file lớn, batch insert, telemetry.



Server Implementation — BatchCreate

grpc-core/server/main.go (thêm method)
// BatchCreate — Client Streaming: nhận nhiều user, trả về tổng kết
func (s *userServer) BatchCreate(stream pb.UserService BatchCreateServer) error {
created := 0
var lastUser *pb.User
// Nhận từng request từ client trong vòng lặp
for {
req, err := stream.Recv()
if err == io.EOF {
// Client đã gửi xong (gọi CloseSend)
break
}
if err != nil {
return status.Errorf(codes.Internal, "lỗi nhận stream: %v", err)
}
// Xử lý từng request
s.mu.Lock()
user := &pb.User{
Id: s.nextID,
Name: req.Name,
Email: req.Email,
Role: pb.Role_ROLE_USER,
}
s.users[s.nextID] = user
s.nextID++
s.mu.Unlock()
lastUser = user
created++
log.Printf("Batch: đã tạo user %d/%s", user.Id, user.Name)
}
// Sau khi nhận hết, gửi 1 response duy nhất
// Ở đây trả về user cuối cùng làm ví dụ
if lastUser == nil {
return status.Error(codes.InvalidArgument, "không có user nào được gửi")
}
log.Printf("Batch hoàn thành: đã tạo %d users", created)
return stream.SendAndClose(lastUser)
}

Client Implementation — Gửi stream

Thêm vào `grpc-core/client/streaming.go`

```
func demoClientStream(client pb.UserServiceClient) {
    ctx, cancel := context.WithTimeout(context.Background(), 15*time.Second)
    defer cancel()

    // Mở stream đến server
    stream, err := client.BatchCreate(ctx)
    if err != nil {
        log.Fatalf("Không thể mở stream: %v", err)
    }

    // Gửi nhiều request — giống upload file từng chunk
    users := []struct{ name, email string }{
        {"Nguyen Van An", "an@example.com"},
        {"Tran Thi Bich", "bich@example.com"},
        {"Le Van Cuong", "cuong@example.com"},
    }

    for _, u := range users {
        fmt.Printf("Gửi: %s\n", u.name)
        if err := stream.Send(&pb.CreateUserRequest{
            Name: u.name,
            Email: u.email,
        }); err != nil {
            log.Fatalf("Lỗi gửi: %v", err)
        }
    }

    // CloseAndRecv = báo đã gửi xong + nhận response
    result, err := stream.CloseAndRecv()
    if err != nil {
        log.Fatalf("Lỗi nhận kết quả: %v", err)
    }
    fmt.Printf("Batch hoàn tất. User cuối: %+v\n", result)
}
```

** Tóm tắt**

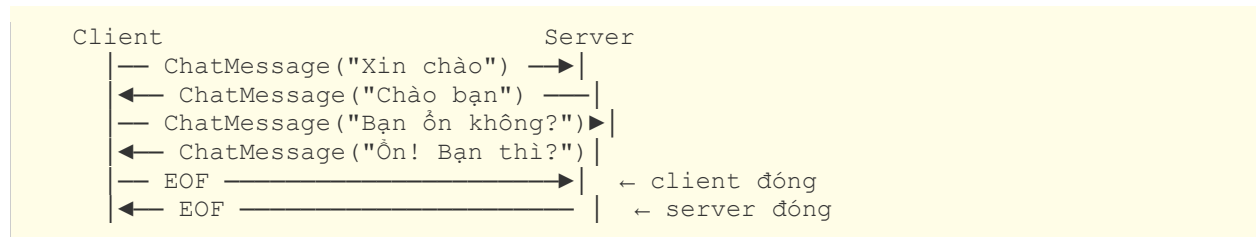
- Server dùng vòng lặp + `stream.Recv()` để nhận từng request từ client
- Client dùng `stream.Send()` nhiều lần, rồi `stream.CloseAndRecv()` để đóng và nhận response
- Server dùng `stream.SendAndClose(response)` để gửi response duy nhất và đóng stream
- `io.EOF` trong `stream.Recv()` = client đã gọi `CloseSend()` — không phải lỗi

** Bài tập tự luyện**

11. Sửa `BatchCreate` để trả về tổng số user đã tạo thay vì user cuối cùng (cần thêm message `BatchResult`).
12. Viết client streaming upload giả lập: đọc danh sách từ file CSV và gửi từng dòng lên server.
13. Nếu client gửi 0 request rồi `CloseAndRecv()`, server nên xử lý thế nào?

2.6 — Bidirectional Streaming RPC

Bidirectional (hai chiều): cả client và server đều có thể gửi nhiều message bất cứ lúc nào, độc lập nhau. Đây là loại mạnh nhất, dùng cho: chat, event sync, real-time collaboration.



Server Implementation — Chat

grpc-core/server/main.go (thêm method)
// Chat — Bidirectional Streaming: cả 2 bên gửi/nhận cùng lúc
func (s *userServer) Chat(stream pb.UserService_ChatServer) error {
// Vòng lặp nhận message từ client
for {
msg, err := stream.Recv()
if err == io.EOF {
// Client đóng stream
return nil
}
if err != nil {
return status.Errorf(codes.Internal, "lỗi nhận: %v", err)
}
log.Printf("Nhận từ user %d: %s", msg.FromId, msg.Text)
// Server gửi phản hồi ngay — không cần đợi hết vòng lặp
reply := &pb.ChatMessage{
FromId: 0, // 0 = server
Text: fmt.Sprintf("Echo: %s", msg.Text),
}
if err := stream.Send(reply); err != nil {
return err
}
}
}

Client Implementation — Gửi và nhận đồng thời

Thêm vào grpc-core/client/streaming.go
func demoBidiStream(client pb.UserServiceClient) {
ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer cancel()
stream, err := client.Chat(ctx)
if err != nil {
log.Fatalf("Không thể mở chat: %v", err)
}
// Goroutine riêng để nhận message từ server
// Hai chiều hoạt động độc lập — đây là điểm khác biệt với REST
done := make(chan struct{})
go func() {
defer close(done)

```

    for {
        msg, err := stream.Recv()
        if err == io.EOF {
            return
        }
        if err != nil {
            log.Printf("Lỗi nhận: %v", err)
            return
        }
        fmt.Printf("Server: %s\n", msg.Text)
    }
}()

// Main goroutine: gửi message đến server
messages := []string{"Xin chào!", "gRPC thật tuyệt!", "Tạm biệt!"}
for _, text := range messages {
    fmt.Printf("Client gửi: %s\n", text)
    if err := stream.Send(&pb.ChatMessage{FromId: 1, Text: text}); err != nil {
        log.Fatalf("Lỗi gửi: %v", err)
    }
    time.Sleep(500 * time.Millisecond)
}

// Báo client xong, đợi server đóng
stream.CloseSend()
<-done
}

```

💡 Pattern quan trọng

Trong Bidirectional streaming, luôn dùng một goroutine riêng để receive và goroutine chính để send. Đây là pattern phổ biến nhất vì send và receive độc lập nhau.

🔗 Tóm tắt

- Bidirectional = cả server lẫn client đều có stream.Send() và stream.Recv()
- Dùng goroutine riêng cho receive để send và receive không block nhau
- Client gọi CloseSend() để báo done nhưng vẫn có thể Recv() tiếp
- Server return nil để đóng stream phía server; client nhận io.EOF

🔧 Bài tập tự luyện

14. Mở rộng Chat để server broadcast message đến TẤT CẢ client đang kết nối (dùng map + mutex để quản lý danh sách stream).
15. Thêm heartbeat: server gửi ping mỗi 5 giây, client trả pong. Nếu client không pong trong 10 giây, server ngắt kết nối.
16. Giải thích sự khác biệt giữa stream.CloseSend() và cancel() context.

2.7 — Generated Code trong Go — Hiểu để dùng đúng

Sau khi chạy protoc, bạn nhận được 2 file. Hiểu nội dung chúng giúp bạn tránh nhầm lẫn và debug hiệu quả hơn.

user.pb.go — Message types

Nội dung quan trọng trong user.pb.go	
// Generated structs — tương đương message trong .proto	
type User struct {	
Id	int32 `protobuf:"varint,1,..." json:"id,omitempty"``
Name	string `protobuf:"bytes,2,..." json:"name,omitempty"``
Email	string `protobuf:"bytes,3,..." json:"email,omitempty"``
Role	Role `protobuf:"varint,4,..." json:"role,omitempty"``
// ... các field quản lý nội bộ protobuf	
}	
// Getter methods (thread-safe với nil receiver)	
func (x *User) GetId()	int32 { if x != nil { return x.Id }; return 0 }
func (x *User) GetName()	string { if x != nil { return x.Name }; return "" }
// Enum values	
type Role int32	
const (	
Role_ROLE_UNSPECIFIED	Role = 0
Role_ROLE_USER	Role = 1
Role_ROLE_ADMIN	Role = 2
)	

user_grpc.pb.go — Service interfaces

Nội dung quan trọng trong user_grpc.pb.go
// — CLIENT SIDE —
// UserServiceClient là interface client sử dụng
type UserServiceClient interface {
GetUser(ctx context.Context, in *GetUserRequest, opts ...grpc.CallOption)
(*User, error)
CreateUser(ctx context.Context, in *CreateUserRequest, opts ...grpc.CallOption)
(*User, error)
ListUsers(ctx context.Context, in *ListUsersRequest, opts ...grpc.CallOption)
(UserService_ListUsersClient, error)
BatchCreate(ctx context.Context, opts ...grpc.CallOption)
(UserService_BatchCreateClient, error)
Chat(ctx context.Context, opts ...grpc.CallOption) (UserService_ChatClient,
error)
}
// Constructor – bạn dùng cái này
func NewUserServiceClient(cc grpc.ClientConnInterface) UserServiceClient
// — SERVER SIDE —
// UserServiceServer là interface bạn phải IMPLEMENT
type UserServiceServer interface {
GetUser(context.Context, *GetUserRequest) (*User, error)
CreateUser(context.Context, *CreateUserRequest) (*User, error)
ListUsers(*ListUsersRequest, UserService_ListUsersServer) error
BatchCreate(UserService_BatchCreateServer) error
Chat(UserService_ChatServer) error
mustEmbedUnimplementedUserServiceServer()

}
// UnimplementedUserServiceServer – nhúng vào struct của bạn
type UnimplementedUserServiceServer struct{}
// Tự động trả UNIMPLEMENTED cho tất cả methods chưa override

Makefile cập nhật

Thêm vào Makefile
Generate cho grpc-core
proto-core:
protoc \
--proto_path=grpc-core/proto \
--go_out=grpc-core/gen \
--go_opt=paths=source_relative \
--go-grpc_out=grpc-core/gen \
--go-grpc_opt=paths=source_relative \
grpc-core/proto/user/user.proto

Tóm tắt

- user.pb.go chứa struct types, enums, serialization/deserialization code
- user_grpc.pb.go chứa Client interface, Server interface và constructor functions
- Bạn implement UserServiceServer interface — không sửa generated files
- UnimplementedUserServiceServer giúp code compile được kể cả khi chưa implement đủ methods

2.8 — Triển khai gRPC Server hoàn chỉnh

Một gRPC server production-ready cần nhiều hơn là `grpc.NewServer()`. Hãy xem các tùy chọn quan trọng.

Server với cấu hình đầy đủ

<code>grpc-core/server/server.go</code>
<code>package main</code>
<code>import (</code>
<code> "fmt"</code>
<code> "log"</code>
<code> "net"</code>
<code> "os"</code>
<code> "os/signal"</code>
<code> "syscall"</code>
<code> "time"</code>
<code>)</code>
<code>pb "grpc-prereqs/grpc-core/gen/user"</code>
<code>"google.golang.org/grpc"</code>
<code>"google.golang.org/grpc/keepalive"</code>
<code>"google.golang.org/grpc/reflection"</code>
<code>)</code>
<code>func runServer() error {</code>
<code> lis, err := net.Listen("tcp", ":50051")</code>
<code> if err != nil {</code>
<code> return fmt.Errorf("listen thất bại: %w", err)</code>
<code> }</code>
<code> // Cấu hình server với các options quan trọng</code>
<code> grpcServer := grpc.NewServer(</code>
<code> // Keepalive: giữ kết nối sống – sẽ giải thích ở mục 2.17</code>
<code> grpc.KeepaliveParams(keepalive.ServerParameters{</code>
<code> MaxConnectionIdle: 15 * time.Minute,</code>
<code> MaxConnectionAge: 30 * time.Minute,</code>
<code> MaxConnectionAgeGrace: 5 * time.Second,</code>
<code> Time: 5 * time.Second,</code>
<code> Timeout: 1 * time.Second,</code>
<code> }),</code>
<code> // Giới hạn kích thước message (bảo vệ khỏi DoS)</code>
<code> grpc.MaxRecvMsgSize(10 * 1024 * 1024), // 10 MB</code>
<code> grpc.MaxSendMsgSize(10 * 1024 * 1024), // 10 MB</code>
<code>)</code>
<code> // Đăng ký service</code>
<code> pb.RegisterUserServiceServer(grpcServer, newUserServer())</code>
<code> // Bật reflection để dùng grpcurl (development only!)</code>
<code> reflection.Register(grpcServer)</code>
<code> // Graceful shutdown khi nhận SIGTERM/SIGINT</code>
<code> quit := make(chan os.Signal, 1)</code>
<code> signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)</code>
<code> go func() {</code>
<code> <-quit</code>
<code> log.Println("Đang tắt server...")</code>
<code> // GracefulStop: đợi requests đang xử lý hoàn thành</code>
<code> grpcServer.GracefulStop()</code>
<code> }()</code>
<code> log.Println("gRPC Server tại :50051")</code>

```
    return grpcServer.Serve(lis)
}

func main() {
    if err := runServer(); err != nil {
        log.Fatal(err)
    }
}
```

Graceful Shutdown vs Stop

Method	Hành vi	Dùng khi
grpcServer.Stop()	Dừng ngay lập tức — ngắt tất cả connections	Khẩn cấp, crash recovery
grpcServer.GracefulStop()	Đợi requests hiện tại hoàn thành, từ chối request mới	Deployment bình thường

Tóm tắt

- `grpc.MaxRecvMsgSize` và `MaxSendMsgSize` bảo vệ server khỏi message quá lớn
- `reflection.Register()` bật gRPC reflection — CHỈ dùng trong development
- `GracefulStop()` đảm bảo requests đang xử lý hoàn thành trước khi shutdown
- Keepalive params giúp detect và dọn dẹp kết nối idle/chết

2.9 — Triển khai gRPC Client hoàn chỉnh

Client cần quản lý connection đúng cách: tái sử dụng, retry, timeout, và đóng kết nối khi không còn dùng.

Connection Management

grpc-core/client/client.go
package main
import (
"context"
"log"
"time"
pb "grpc-prereqs/grpc-core/gen/user"
"google.golang.org/grpc"
"google.golang.org/grpc/credentials/insecure"
"google.golang.org/grpc/keepalive"
)
// UserClient là wrapper giúp quản lý connection tập trung
type UserClient struct {
conn *grpc.ClientConn
client pb.UserServiceClient
}
// NewUserClient tạo kết nối với cấu hình đầy đủ
func NewUserClient(addr string) (*UserClient, error) {
conn, err := grpc.NewClient(
addr,
grpc.WithTransportCredentials(insecure.NewCredentials()),
// Keepalive client – ping server để giữ kết nối sống
grpc.WithKeepaliveParams(keepalive.ClientParameters{
Time: 10 * time.Second,
Timeout: 3 * time.Second,
PermitWithoutStream: true, // Ping kể cả khi không có stream
}),
// Wait for Ready: đợi kết nối thay vì fail ngay
grpc.WithBlock(), // Chặn cho đến khi kết nối thành công
)
if err != nil {
return nil, err
}
return &UserClient{
conn: conn,
client: pb.NewUserServiceClient(conn),
}, nil
}
// Close đóng kết nối – luôn gọi khi không dùng nữa
func (uc *UserClient) Close() error {
return uc.conn.Close()
}
// GetUser là wrapper method – ẩn stub khỏi caller
func (uc *UserClient) GetUser(ctx context.Context, id int32) (*pb.User, error) {
return uc.client.GetUser(ctx, &pb.GetUserRequest{Id: id})
}
func main() {
// Tạo context với timeout kết nối
ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()

<code>uc, err := NewUserClient("localhost:50051")</code>
<code>if err != nil {</code>
<code> log.Fatalf("Không thể kết nối: %v", err)</code>
<code>}</code>
<code>defer uc.Close() // QUAN TRỌNG: luôn đóng connection</code>
<code>// Dùng client</code>
<code>user, err := uc.GetUser(ctx, 1)</code>
<code>if err != nil {</code>
<code> log.Printf("Lỗi: %v", err)</code>
<code> return</code>
<code>}</code>
<code>log.Printf("Kết quả: %+v", user)</code>
<code>}</code>

💡 Tái sử dụng Connection

Một `*grpc.ClientConn` hỗ trợ nhiều goroutine và nhiều RPC call đồng thời. KHÔNG tạo connection mới cho mỗi request — tốn tài nguyên và chậm. Tạo 1 lần, dùng suốt vòng đời chương trình.

📌 Tóm tắt

- `grpc.NewClient()` thay thế `grpc.Dial()` từ v1.64 — dùng `NewClient`
- Bọc connection trong struct để ẩn chi tiết và tái sử dụng dễ hơn
- Luôn `defer conn.Close()` để giải phóng resource
- `grpc.WithBlock()` giúp đợi kết nối thay vì fail ngay khi server chưa sẵn sàng

2.10 — Deadline và Timeout

Trong hệ thống phân tán, một request không nên đợi vô tận. Deadline và Timeout là cơ chế bảo vệ quan trọng nhất.

Sự khác biệt: Timeout vs Deadline

	Timeout	Deadline
Định nghĩa	Khoảng thời gian từ lúc bắt đầu	Thời điểm cụ thể phải xong
Ví dụ	"Tối đa 5 giây"	"Phải xong trước 15:30:00"
Trong Go	<code>context.WithTimeout(ctx, 5s)</code>	<code>context.WithDeadline(ctx, time.Now().Add(5s))</code>
Propagation	Cần tính lại khi truyền xuống	Truyền nguyên — tự giảm

Deadline propagation — tại sao dùng Deadline thay Timeout?

```
Service A (timeout=10s) → Service B → Service C
```

Vấn đề với Timeout:
A: timeout=10s → B: timeout=10s → C: timeout=10s (tổng 30s!)

Đúng với Deadline:
A: deadline=T+10s → B: deadline=T+10s → C: deadline=T+10s (tổng 10s)
(deadline được truyền nguyên qua metadata, tự giảm dần)

Ví dụ thực tế

<code>grpc-core/client/deadline.go</code>
<code>package main</code>
<code>import (</code>
<code> "context"</code>
<code> "fmt"</code>
<code> "log"</code>
<code> "time"</code>
<code>)</code>
<code>pb "grpc-prereqs/grpc-core/gen/user"</code>
<code>"google.golang.org/grpc/codes"</code>
<code>"google.golang.org/grpc/status"</code>
<code>)</code>
<code>func demoDeadline(client pb.UserServiceClient) {</code>
<code> // — Ví dụ 1: Timeout bình thường —</code>
<code> ctx1, cancel1 := context.WithTimeout(context.Background(), 5*time.Second)</code>
<code> defer cancel1()</code>
<code> user, err := client.GetUser(ctx1, &pb.GetUserRequest{Id: 1})</code>
<code> if err != nil {</code>
<code> // Phân biệt deadline exceeded vs lỗi khác</code>
<code> st, _ := status.FromError(err)</code>
<code> switch st.Code() {</code>
<code> case codes.DeadlineExceeded:</code>
<code> log.Println("Request quá hạn! Server không đủ nhanh hoặc mạng chậm.")</code>
<code> case codes.Canceled:</code>
<code> log.Println("Request bị hủy trước deadline.")</code>

default:
log.Printf("Lỗi khác: %v", err)
}
return
}
fmt.Printf("Nhận user: %+v\n", user)
// — Ví dụ 2: Timeout ngắn để thấy DeadlineExceeded —
ctx2, cancel2 := context.WithTimeout(context.Background(), 1*time.Nanosecond)
defer cancel2()
, err = client.GetUser(ctx2, &pb.GetUserRequest{Id: 1})
if err != nil {
st, _ := status.FromError(err)
fmt.Printf("Timeout ngắn → %s: %s\n", st.Code(), st.Message())
}
}
// Server cũng nên kiểm tra deadline
func (s *userServer) GetUserWithDeadlineCheck(
ctx context.Context, req *pb.GetUserRequest) (*pb.User, error) {
// Kiểm tra deadline trước khi làm việc nặng
if deadline, ok := ctx.Deadline(); ok {
remaining := time.Until(deadline)
if remaining < 100*time.Millisecond {
return nil, status.Error(codes.DeadlineExceeded,
"không đủ thời gian để xử lý")
}
log.Printf("Còn %.2f giây để xử lý", remaining.Seconds())
}
// Xử lý bình thường...
s.mu.RLock()
defer s.mu.RUnlock()
user, ok := s.users[req.Id]
if !ok {
return nil, status.Errorf(codes.NotFound, "user %d không tồn tại", req.Id)
}
return user, nil
}

Tóm tắt

- Luôn đặt timeout cho mọi gRPC call — không bao giờ để request treo vô tận
- Deadline (thời điểm cố định) tốt hơn Timeout trong chuỗi service vì tự propagate
- codes.DeadlineExceeded = hết hạn; codes.Canceled = bị hủy chủ động
- Server nên kiểm tra ctx.Deadline() trước khi bắt đầu xử lý tốn thời gian

Bài tập tự luyện

- Viết một server method cố tình sleep 2 giây. Test với client timeout 1s và 3s.
- Khi context bị cancel, request đang xử lý trong server có bị dừng không? Giải thích.
- Tại sao codes.DeadlineExceeded và codes.Canceled lại là 2 code khác nhau?

2.11 — Context Propagation

context.Context là cơ chế truyền thông tin xuyên suốt call stack: deadline, cancellation signal, và các giá trị tùy chỉnh (trace ID, auth token).

Context mang những gì?

Thông tin	Cách thêm	Cách đọc
Deadline/Timeout	context.WithDeadline() / WithTimeout()	ctx.Deadline()
Cancellation	context.WithCancel()	ctx.Done() channel
Custom values	context.WithValue(ctx, key, val)	ctx.Value(key)
Metadata (gRPC)	metadata.NewOutgoingContext()	metadata.FromIncomingContext()

Truyền context trong service chain

<code>grpc-core/server/context_demo.go</code>
<code>package main</code>
<code>import (</code>
<code> "context"</code>
<code> "log"</code>
<code>)</code>
<code>pb "grpc-prereqs/grpc-core/gen/user"</code>
<code>"google.golang.org/grpc/codes"</code>
<code>"google.golang.org/grpc/status"</code>
<code>)</code>
<code>// Key type riêng để tránh xung đột với package khác</code>
<code>type contextKey string</code>
<code>const (</code>
<code> requestIDKey contextKey = "request_id"</code>
<code> callerKey contextKey = "caller"</code>
<code>)</code>
<code>// GetUserWithContext – truyền và đọc context values</code>
<code>func (s *userServer) GetUserWithContext(</code>
<code> ctx context.Context, req *pb.GetUserRequest) (*pb.User, error) {</code>
<code> // Kiểm tra context còn sống không</code>
<code> select {</code>
<code> case <-ctx.Done():</code>
<code> return nil, status.Error(codes.Canceled, "request bị hủy")</code>
<code> default:</code>
<code> // Context còn sống, tiếp tục</code>
<code> }</code>
<code>// Đọc request ID từ context (được set bởi interceptor)</code>
<code>if reqID, ok := ctx.Value(requestIDKey).(string); ok {</code>
<code> log.Printf("[%s] GetUser id=%d", reqID, req.ID)</code>
<code>}</code>
<code>// Khi gọi service khác, PHẢI truyền ctx – không dùng context.Background()</code>
<code>// ví dụ: otherService.DoSomething(ctx, ...) ✓</code>
<code>// ví dụ: otherService.DoSomething(context.Background(), ...) ✗ Mất deadline!</code>

```
s.mu.RLock()
defer s.mu.RUnlock()

user, ok := s.users[req.Id]
if !ok {
    return nil, status.Errorf(codes.NotFound, "user %d không tồn tại", req.Id)
}
return user, nil
}
```

⚠ Anti-pattern phổ biến

KHÔNG bao giờ gọi downstream service với `context.Background()` khi đã có ctx từ caller. Điều này phá vỡ deadline propagation — downstream service sẽ không biết phải dừng khi nào.

🔗 Tóm tắt

- Luôn truyền ctx từ parameter xuống tất cả function calls — không tự tạo `context.Background()`
- Dùng type alias cho context key để tránh xung đột giữa các packages
- `ctx.Done()` là channel — nhận tín hiệu khi context bị cancel hoặc timeout
- Deadline, cancel signal, và values đều tự động propagate khi truyền ctx

2.12 — Metadata trong gRPC

Metadata là key-value pairs được gửi kèm theo request/response, tương đương HTTP headers. Dùng cho: auth token, request ID, tracing, API version.

Gửi Metadata từ Client

<code>grpc-core/client/metadata_demo.go</code>
<code>package main</code>
<code>import (</code>
<code> "context"</code>
<code> "fmt"</code>
<code> "log"</code>
<code> pb "grpc-prereqs/grpc-core/gen/user"</code>
<code> "google.golang.org/grpc/metadata"</code>
<code>)</code>
<code>func demoMetadata(client pb.UserServiceClient) {</code>
<code> // Tạo metadata - tương đương HTTP request headers</code>
<code> md := metadata.Pairs(</code>
<code> "authorization", "Bearer my-jwt-token-123",</code>
<code> "x-request-id", "req-abc-456",</code>
<code> "x-caller-service", "api-gateway",</code>
<code>)</code>
<code> // Gắn metadata vào context - gửi kèm theo mọi request</code>
<code> ctx := metadata.NewOutgoingContext(context.Background(), md)</code>
<code> // Nhận header và trailer từ server</code>
<code> var header, trailer metadata.MD</code>
<code> user, err := client.GetUser(</code>
<code> ctx,</code>
<code> &pb.GetUserRequest{Id: 1},</code>
<code> // grpc.Header và grpc.Trailer để nhận metadata từ server</code>
<code> // grpc.Header(&header), grpc.Trailer(&trailer),</code>
<code>)</code>
<code> if err != nil {</code>
<code> log.Printf("Lỗi: %v", err)</code>
<code> return</code>
<code> }</code>
<code> fmt.Printf("User: %v\n", user)</code>
<code> fmt.Printf("Response header: %v\n", header)</code>
<code> fmt.Printf("Response trailer: %v\n", trailer)</code>
<code>}</code>

Đọc Metadata ở Server

<code>grpc-core/server/metadata_demo.go</code>
<code>package main</code>
<code>import (</code>
<code> "context"</code>
<code> "log"</code>
<code> pb "grpc-prereqs/grpc-core/gen/user"</code>
<code> "google.golang.org/grpc/codes"</code>

```

"google.golang.org/grpc/metadata"
"google.golang.org/grpc/status"
)

func (s *userServer) GetUserWithAuth(
    ctx context.Context, req *pb.GetUserRequest) (*pb.User, error) {

    // Đọc incoming metadata
    md, ok := metadata.FromIncomingContext(ctx)
    if !ok {
        return nil, status.Error(codes.Unauthenticated, "thiếu metadata")
    }

    // Lấy authorization header
    authValues := md.Get("authorization")
    if len(authValues) == 0 {
        return nil, status.Error(codes.Unauthenticated, "thiếu authorization token")
    }
    token := authValues[0]
    log.Printf("Token nhận được: %s", token)

    // Lấy request ID (để logging/tracing)
    if reqIDs := md.Get("x-request-id"); len(reqIDs) > 0 {
        log.Printf("Request ID: %s", reqIDs[0])
    }

    // Gửi metadata về client (response header)
    header := metadata.Pairs("x-processed-by", "user-service-v1")
    // grpc.SendHeader(ctx, header) // Gửi header ngay
    // = header // suppress unused warning trong ví dụ

    s.mu.RLock()
    defer s.mu.RUnlock()
    user, ok := s.users[req.Id]
    if !ok {
        return nil, status.Errorf(codes.NotFound, "user %d không tồn tại", req.Id)
    }
    return user, nil
}

```

Header vs Trailer

	Header Metadata	Trailer Metadata
Khi gửi	Trước khi gửi body	Sau khi gửi xong body
Dùng cho	Auth, request ID, routing	Status, timing, result metadata
gRPC API	grpc.SendHeader(ctx, md)	grpc.SetTrailer(ctx, md)
Client nhận	Ngay khi nhận response	Sau khi stream kết thúc

Tóm tắt

- Metadata = HTTP headers trong gRPC — key lowercase, value string/binary
- Client dùng metadata.NewOutgoingContext(); Server dùng metadata.FromIncomingContext()
- Header gửi trước body; Trailer gửi sau khi body hoàn tất

- Dùng metadata cho: auth token, request ID, API version, tracing headers

Bài tập tự luyện

20. Viết interceptor (mục 2.14) đọc x-request-id từ metadata và thêm vào log của mỗi request.
21. Tại sao metadata key phải viết thường (lowercase)? Điều gì xảy ra nếu dùng uppercase?

2.13 — Error Handling và Status Codes

gRPC dùng hệ thống status code chuẩn hóa — không phải HTTP 400/404/500. Dùng đúng code giúp client xử lý lỗi chính xác.

Các Status Code quan trọng nhất

Code	Giá trị số	Ý nghĩa	Dùng khi
OK	0	Thành công	Không trả lỗi — trả nil
CANCELED	1	Request bị hủy	Client cancel, context done
UNKNOWN	2	Lỗi không rõ	Lỗi không có category phù hợp
INVALID_ARGUMENT	3	Tham số không hợp lệ	Validation failed, bad input
DEADLINE_EXCEEDED	4	Quá thời hạn	Timeout, deadline passed
NOT_FOUND	5	Không tìm thấy	Entity không tồn tại
ALREADY_EXISTS	6	Đã tồn tại	Duplicate create
PERMISSION_DENIED	7	Không có quyền	Authenticated nhưng thiếu quyền
RESOURCE_EXHAUSTED	8	Hết tài nguyên	Rate limit, quota exceeded
INTERNAL	13	Lỗi internal	Lỗi server không thể phân loại
UNAVAILABLE	14	Service không khả dụng	Retry được — server đang bận
UNAUTHENTICATED	16	Chưa xác thực	Thiếu hoặc sai auth credentials

Tạo và trả lỗi đúng cách

grpc-core/server/errors.go
package main
import (
"context"
"fmt"
)
pb "grpc-prereqs/grpc-core/gen/user"
"google.golang.org/grpc/codes"
"google.golang.org/grpc/status"
)
// — Tạo lỗi có status code —
// Cách 1: status.Error — đơn giản
func errSimple() error {
return status.Error(codes.NotFound, "user không tồn tại")
}
// Cách 2: status.Errorf — có format string
func errFormatted(id int32) error {
return status.Errorf(codes.NotFound, "user id=%d không tồn tại", id)
}
// Cách 3: status.New + WithDetails — thêm thông tin chi tiết
func errWithDetails(id int32) error {

st := status.New(codes.NotFound, fmt.Sprintf("user %d không tồn tại", id))
// Có thể attach proto message làm detail
// st, _ = st.WithDetails(&pb.User{Id: id})
return st.Err()
}
// — Đọc lỗi từ phía client —
func handleGRPCError(client pb.UserServiceClient) {
ctx := context.Background()
, err := client.GetUser(ctx, &pb.GetUserRequest{Id: 9999})
if err != nil {
// Chuyển error thành *status.Status để đọc code
st, ok := status.FromError(err)
if !ok {
// Không phải gRPC error (network error, etc.)
fmt.Printf("Non-gRPC error: %v\n", err)
return
}
// Switch theo status code
switch st.Code() {
case codes.NotFound:
fmt.Printf("Không tìm thấy: %s\n", st.Message())
case codes.InvalidArgument:
fmt.Printf("Input không hợp lệ: %s\n", st.Message())
case codes.Unauthenticated:
fmt.Printf("Chưa đăng nhập: %s\n", st.Message())
case codes.DeadlineExceeded:
fmt.Printf("Request quá hạn — cần retry\n")
case codes.Unavailable:
fmt.Printf("Server đang bận — nên retry sau\n")
default:
fmt.Printf("Lỗi [%s]: %s\n", st.Code(), st.Message())
}
}
}

Lỗi nào có thể Retry?

Code	Retry?	Lý do
UNAVAILABLE	Có (với backoff)	Server tạm thời bận
DEADLINE_EXCEEDED	Có (cẩn thận)	Có thể thành công nếu retry nhanh
INTERNAL	Đôi khi	Phụ thuộc vào server logic
NOT_FOUND	Không	Data không tồn tại — retry vô ích
INVALID_ARGUMENT	Không	Input sai — phải sửa input
UNAUTHENTICATED	Không (trừ refresh token)	Phải lấy token mới

Tóm tắt

- Dùng status.Errorf(codes.X, ...) thay vì fmt.Errorf() để gRPC client đọc được code
- Client dùng status.FromError(err) để lấy status code và message

- Chọn code phù hợp: NOT_FOUND, INVALID_ARGUMENT, UNAUTHENTICATED, PERMISSION_DENIED...
- UNAVAILABLE và DEADLINE_EXCEEDED có thể retry; INVALID_ARGUMENT và NOT_FOUND thì không

Bài tập tự luyện

22. Viết function validateCreateUserRequest kiểm tra name không rỗng, email có @ — trả InvalidArgument nếu sai.
23. Tại sao UNAUTHENTICATED (16) và PERMISSION_DENIED (7) là 2 code khác nhau? Cho ví dụ cụ thể mỗi loại.
24. Implement retry logic: gọi GetUser tối đa 3 lần nếu gặp UNAVAILABLE, dừng ngay nếu NOT_FOUND.

2.14 — Interceptors (Middleware trong gRPC)

Interceptor là middleware của gRPC — chặn mọi request/response để thực thi cross-cutting logic: logging, auth, metrics, recovery mà không lặp code trong từng handler.

Hai loại Interceptor

Loại	Áp dụng cho	Interface
Unary Interceptor	Unary RPC (1 req → 1 resp)	grpc.UnaryServerInterceptor
Stream Interceptor	Streaming RPC (mọi loại stream)	grpc.StreamServerInterceptor

Logging Interceptor

<code>grpc-core/server/interceptors.go</code>
<code>package main</code>
<code>import (</code>
<code> "context"</code>
<code> "log"</code>
<code> "time"</code>
<code> "google.golang.org/grpc"</code>
<code> "google.golang.org/grpc/codes"</code>
<code> "google.golang.org/grpc/status"</code>
<code>)</code>
<code>// loggingInterceptor ghi log thời gian và kết quả mỗi Unary RPC</code>
<code>func loggingInterceptor(</code>
<code> ctx context.Context,</code>
<code> req interface{},</code>
<code> info *grpc.UnaryServerInfo, // Chứa tên method: /user.UserService/GetUser</code>
<code> handler grpc.UnaryHandler, // Handler thực sự (logic của bạn)</code>
<code>) (interface{}, error) {</code>
<code> start := time.Now()</code>
<code> // Gọi handler tiếp theo (hoặc interceptor tiếp theo trong chain)</code>
<code> resp, err := handler(ctx, req)</code>
<code> duration := time.Since(start)</code>
<code> code := codes.OK</code>
<code> if err != nil {</code>
<code> st, _ := status.FromError(err)</code>
<code> code = st.Code()</code>
<code> }</code>
<code> log.Printf("[gRPC] method=%s duration=%v code=%s",</code>
<code> info.FullMethod, duration, code)</code>
<code> return resp, err</code>
<code>}</code>
<code>// recoveryInterceptor bắt panic và chuyển thành INTERNAL error</code>
<code>func recoveryInterceptor(</code>
<code> ctx context.Context,</code>
<code> req interface{},</code>
<code> info *grpc.UnaryServerInfo,</code>
<code> handler grpc.UnaryHandler,</code>
<code>) (resp interface{}, err error) {</code>
<code> // defer recover để bắt panic</code>

defer func() {
if r := recover(); r != nil {
log.Printf("PANIC recovered in %s: %v", info.FullMethod, r)
err = status.Errorf(codes.Internal, "server gặp lỗi nội bộ")
}
}()
return handler(ctx, req)
}
// authInterceptor kiểm tra authorization token từ metadata
func authInterceptor(
ctx context.Context,
req interface{},
info *grpc.UnaryServerInfo,
handler grpc.UnaryHandler,
) (interface{}, error) {
// Bỏ qua auth cho các method public
publicMethods := map[string]bool{
"/user.UserService/CreateUser": true,
}
if publicMethods[info.FullMethod] {
return handler(ctx, req)
}
// Đọc token từ metadata
md, ok := metadata.FromIncomingContext(ctx)
if !ok {
return nil, status.Error(codes.Unauthenticated, "thiếu metadata")
}
tokens := md.Get("authorization")
if len(tokens) == 0 tokens[0] == "" {
return nil, status.Error(codes.Unauthenticated, "thiếu token")
}
// Validate token (giả lập)
if tokens[0] != "Bearer valid-token" {
return nil, status.Error(codes.Unauthenticated, "token không hợp lệ")
}
return handler(ctx, req)
}

Đăng ký Interceptors với Server

grpc-core/server/server.go – cập nhật
// Chain nhiều interceptors – thứ tự quan trọng!
// Request đi qua: recovery → logging → auth → handler
// Response đi ngược: handler → auth → logging → recovery
grpcServer := grpc.NewServer(
grpc.ChainUnaryInterceptor(
recoveryInterceptor, // Ngoài cùng – bắt tất cả panic
loggingInterceptor, // Ghi log kể cả khi auth fail
authInterceptor, // Trong cùng – gần handler nhất
),
)

Request flow:

recoveryInterceptor → loggingInterceptor → authInterceptor → handler

Response flow:

←

```
recoveryInterceptor ← loggingInterceptor ← authInterceptor ← handler
```

Client-side Interceptor

grpc-core/client/interceptors.go
// clientLoggingInterceptor ghi log mọi outgoing request
func clientLoggingInterceptor(ctx context.Context, method string, // Tên method: /user.UserService/GetUser req, reply interface{}, cc *grpc.ClientConn, invoker grpc.UnaryInvoker, // Gọi tiếp theo opts ...grpc.CallOption,) error {
start := time.Now()
err := invoker(ctx, method, req, reply, cc, opts...)
log.Printf("[Client] method=%s duration=%v err=%v", method, time.Since(start), err)
return err
}
// Đăng ký với client connection
conn, err := grpc.NewClient("localhost:50051", grpc.WithTransportCredentials(insecure.NewCredentials()), grpc.WithUnaryInterceptor(clientLoggingInterceptor),)

Tóm tắt

- Interceptor = middleware — chạy trước và sau mỗi RPC handler
- grpc.ChainUnaryInterceptor cho phép đặt nhiều interceptor theo thứ tự
- Thứ tự: recoveryInterceptor ngoài cùng, authInterceptor gần handler nhất
- Client cũng có interceptor — dùng grpc.WithUnaryInterceptor()

Bài tập tự luyện

- Viết rateLimit interceptor: từ chối request nếu client gửi quá 10 request/giây (dùng sync.Map + time.Ticker).
- Viết stream interceptor để log tên method của mọi streaming RPC.
- Nếu authInterceptor đặt ngoài cùng và loggingInterceptor đặt trong, request bị reject vì auth sẽ bị log không? Tại sao?

2.15 — Reflection và grpcurl

gRPC Reflection cho phép server expose metadata về services của mình. grpcurl dùng thông tin này để bạn gọi gRPC từ terminal như dùng curl với REST.

Bật gRPC Reflection

grpc-core/server/server.go (đã thêm ở mục 2.8)
import "google.golang.org/grpc/reflection"
// Sau khi đăng ký service:
pb.RegisterUserServiceServer(grpcServer, newUserServer())
// Bật reflection — CHỈ trong development/staging
// KHÔNG bật trong production vì expose service structure
reflection.Register(grpcServer)

Cài grpcurl

Ubuntu/macOS
macOS
brew install grpcurl
Ubuntu
go install github.com/fullstorydev/grpcurl/cmd/grpcurl@latest
Windows (PowerShell)
go install github.com/fullstorydev/grpcurl/cmd/grpcurl@latest
Thêm %USERPROFILE%\go\bin vào PATH

Các lệnh grpcurl thường dùng

terminal
Liệt kê tất cả services
grpcurl -plaintext localhost:50051 list
user.UserService
Liệt kê tất cả methods của UserService
grpcurl -plaintext localhost:50051 list user.UserService
user.UserService.CreateUser
user.UserService.GetUser
user.UserService.ListUsers
Describe message types
grpcurl -plaintext localhost:50051 describe user.User
Gọi Unary RPC
grpcurl -plaintext -d '{"name":"An","email":"an@test.com"}' \
localhost:50051 user.UserService/CreateUser
Gọi với metadata (auth token)
grpcurl -plaintext \
-H "authorization: Bearer valid-token" \
-d '{"id":1}' \
localhost:50051 user.UserService/GetUser

Gọi Server Streaming
grpcurl -plaintext -d '{"limit":5}' \
localhost:50051 user.UserService/ListUsers

💡 grpcui — GUI cho gRPC

grpcui là phiên bản có giao diện web của grpcurl. Cài: go install github.com/fullstorydev/grpcui/cmd/grpcui@latest. Chạy: grpcui -plaintext localhost:50051 → mở browser để test service.

🔗 Tóm tắt

- reflection.Register() bật gRPC Server Reflection — chỉ dùng trong development
- grpcurl cho phép gọi gRPC method từ terminal như curl với REST API
- grpcurl list = liệt kê services; describe = xem schema; -d = request body JSON
- grpcui mở giao diện web để test gRPC service tương tác

🔗 Bài tập tự luyện

28. Bật reflection trên server, dùng grpcurl để gọi CreateUser 3 lần, ListUsers và GetUser.
29. Dùng grpcurl với -H để test authInterceptor từ mục 2.14 — test cả trường hợp token hợp lệ và không hợp lệ.

2.16 — TLS / mTLS

TLS mã hóa đường truyền giữa client và server. mTLS (mutual TLS) bổ sung xác thực hai chiều — cả client lẫn server đều cần certificate hợp lệ.

TLS vs mTLS

	Không TLS	TLS	mTLS
Mã hóa	Không	Có	Có
Server xác thực	Không	Có (certificate)	Có (certificate)
Client xác thực	Không	Không	Có (client cert)
Dùng cho	Development	Public API	Service-to-service

Tạo self-signed certificates (Development)

terminal — Tạo CA và certificates
<code>mkdir -p grpc-core/certs && cd grpc-core/certs</code>
<code># 1. Tạo Certificate Authority (CA)</code>
<code>openssl genrsa -out ca.key 4096</code>
<code>openssl req -new -x509 -key ca.key -sha256 -days 365 \</code>
<code>-out ca.crt -subj "/CN=MyCA"</code>
<code># 2. Tạo Server certificate</code>
<code>openssl genrsa -out server.key 4096</code>
<code>openssl req -new -key server.key -out server.csr -subj "/CN=localhost"</code>
<code>openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key \</code>
<code>-CAcreateserial -out server.crt -days 365 -sha256</code>
<code># 3. Tạo Client certificate (chỉ cần cho mTLS)</code>
<code>openssl genrsa -out client.key 4096</code>
<code>openssl req -new -key client.key -out client.csr -subj "/CN=client"</code>
<code>openssl x509 -req -in client.csr -CA ca.crt -CAkey ca.key \</code>
<code>-CAcreateserial -out client.crt -days 365 -sha256</code>

Server với TLS

grpc-core/server/tls_server.go
<code>package main</code>
<code>import (</code>
<code> "crypto/tls"</code>
<code> "crypto/x509"</code>
<code> "log"</code>
<code> "os"</code>
<code>)</code>
<code>"google.golang.org/grpc"</code>
<code>"google.golang.org/grpc/credentials"</code>
<code>)</code>
<code>func newTLSSEServer() *grpc.Server {</code>
<code> // Đọc server certificate và key</code>
<code> cert, err := tls.LoadX509KeyPair("grpc-core/certs/server.crt", "grpc-</code>
<code>core/certs/server.key")</code>

if err != nil {
log.Fatalf("Không thể load certificate: %v", err)
}
// Đọc CA certificate để verify client (chỉ cần cho mTLS)
caCert, err := os.ReadFile("grpc-core/certs/ca.crt")
if err != nil {
log.Fatalf("Không thể đọc CA cert: %v", err)
}
caPool := x509.NewCertPool()
caPool.AppendCertsFromPEM(caCert)
// Cấu hình TLS cho server
tlsConfig := &tls.Config{
Certificates: []tls.Certificate{cert},
// ClientAuth: tls.RequireAndVerifyClientCert, // Bật để dùng mTLS
// ClientCAs: caPool, // Bật để dùng mTLS
}
creds := credentials.NewTLS(tlsConfig)
return grpc.NewServer(grpc.Creds(creds))
}

Client với TLS

grpc-core/client/tls_client.go
package main
import (
"crypto/tls"
"crypto/x509"
"log"
"os"
"google.golang.org/grpc"
"google.golang.org/grpc/credentials"
)
func newTLSClient(addr string) *grpc.ClientConn {
// Đọc CA certificate để verify server
caCert, err := os.ReadFile("grpc-core/certs/ca.crt")
if err != nil {
log.Fatalf("Không thể đọc CA cert: %v", err)
}
caPool := x509.NewCertPool()
caPool.AppendCertsFromPEM(caCert)
tlsConfig := &tls.Config{
RootCAs: caPool,
ServerName: "localhost", // Phải khớp với CN trong server.crt
// Thêm client certificate để dùng mTLS:
// Certificates: []tls.Certificate{clientCert},
}
creds := credentials.NewTLS(tlsConfig)
conn, err := grpc.NewClient(addr, grpc.WithTransportCredentials(creds))
if err != nil {
log.Fatalf("Không thể kết nối TLS: %v", err)
}
return conn
}

 Tóm tắt

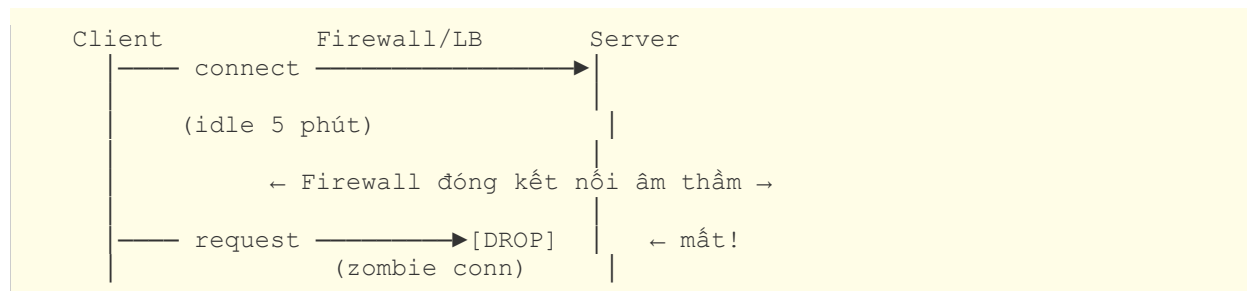
- TLS mã hóa đường truyền — bắt buộc khi chạy production
- mTLS = TLS + client cũng phải có certificate — phù hợp service-to-service
- Self-signed certificate chỉ dùng trong development — production dùng Let's Encrypt hoặc internal CA
- ServerName trong client TLS phải khớp CN trong server certificate

2.17 — Keepalive và Connection Management

Keepalive giúp phát hiện kết nối chết sớm và giữ kết nối sống qua NAT/firewall timeout. Quan trọng cho hệ thống microservices có traffic dài hạn.

Vấn đề: Kết nối "zombie"

Kết nối TCP có thể bị firewall hoặc load balancer đóng âm thầm sau một khoảng thời gian idle. gRPC không biết kết nối đã chết cho đến khi gửi request tiếp theo — gây ra lỗi khó debug.



Cấu hình Keepalive đầy đủ

grpc-core/server/keepalive_server.go

```

package main

import (
    "time"
    "google.golang.org/grpc"
    "google.golang.org/grpc/keepalive"
)

func newServerWithKeepalive() *grpc.Server {
    return grpc.NewServer(
        // — Server Keepalive Parameters —
        grpc.KeepaliveParams(keepalive.ServerParameters{
            // Sau khi kết nối idle bao lâu thì gửi ping
            Time: 2 * time.Minute,
            // Đợi pong trong bao lâu trước khi đóng kết nối
            Timeout: 20 * time.Second,
            // Đóng kết nối đã idle bao lâu
            MaxConnectionIdle: 15 * time.Minute,
            // Đóng kết nối đã sống quá lâu (force reconnect)
            MaxConnectionAge: 2 * time.Hour,
            // Thời gian grace khi đóng MaxConnectionAge
            MaxConnectionAgeGrace: 5 * time.Second,
        }),
        // — Enforcement Policy: chống client ping quá nhiều —
        grpc.KeepaliveEnforcementPolicy(keepalive.EnforcementPolicy{
            // Client phải đợi ít nhất N giây giữa các ping
            MinTime: 5 * time.Second,
            // Cho phép ping kể cả khi không có stream đang mở
            PermitWithoutStream: true,
        }),
    )
}

```

grpc-core/client/keepalive_client.go

```

package main

```

```

import (
    "time"
    "google.golang.org/grpc"
    "google.golang.org/grpc/keepalive"
)

func newClientWithKeepalive(addr string) (*grpc.ClientConn, error) {
    return grpc.NewClient(
        addr,
        grpc.WithKeepaliveParams(keepalive.ClientParameters{
            // Gửi ping sau khi idle N giây
            Time: 10 * time.Second,
            // Đợi pong trong N giây
            Timeout: 3 * time.Second,
            // Cho phép ping kể cả không có stream
            PermitWithoutStream: true,
        }),
    )
}

```

Bảng tổng hợp các Server Parameters

Parameter	Ý nghĩa	Giá trị gợi ý
MaxConnectionIdle	Đóng conn idle quá lâu	15-30 phút
MaxConnectionAge	Force reconnect sau N thời gian	1-2 giờ
MaxConnectionAgeGrace	Grace period trước khi force close	5-30 giây
Time	Tần suất gửi keepalive ping	30-120 giây
Timeout	Đợi pong trước khi đóng	10-30 giây

Tóm tắt

- Keepalive phát hiện kết nối chết trước khi gửi request thực sự — tránh lỗi bí ẩn
- MaxConnectionAge force reconnect định kỳ — giúp load balancing hoạt động đúng
- EnforcementPolicy bảo vệ server khỏi client ping quá dày
- PermitWithoutStream: true cho phép ping khi không có stream đang mở

Bài tập tự luyện

30. Thêm keepalive vào cả server và client từ mục 2.3. Dùng tcpdump (Linux/macOS) để quan sát ping-pong packets.
31. Tại sao MaxConnectionAge giúp load balancing hoạt động đúng hơn? Gợi ý: nghĩ đến trường hợp thêm server node mới.

2.18 — Tổng kết

Bảng ôn tập toàn tài liệu

Mục	Khái niệm	Điều quan trọng nhất
2.1	Kiến trúc gRPC	Stub → Channel → HTTP/2 → Server → Handler
2.2	File .proto	Field number không đổi; đặt tên theo chuẩn
2.3	Unary RPC	UnimplementedServer; status.Errorf cho errors
2.4	Server Streaming	stream.Send() loop; client Recv() đến io.EOF
2.5	Client Streaming	stream.Recv() loop; SendAndClose() response
2.6	Bidirectional	Goroutine riêng cho Recv; CloseSend() cuối
2.7	Generated Code	.pb.go = types; _grpc.pb.go = interfaces
2.8	gRPC Server	GracefulStop(); MaxRecvMsgSize; reflection
2.9	gRPC Client	Tái sử dụng ClientConn; defer Close()
2.10	Deadline/Timeout	Deadline propagate; codes.DeadlineExceeded
2.11	Context Propagation	Luôn truyền ctx — không dùng Background()
2.12	Metadata	Outgoing/Incoming; header vs trailer
2.13	Error Handling	status.Errorf + codes; FromError() ở client
2.14	Interceptors	ChainUnaryInterceptor; recovery ngoài, auth trong
2.15	Reflection	development only; grpcurl để test
2.16	TLS/mTLS	TLS encrypt; mTLS mutual auth; ServerName match
2.17	Keepalive	MaxConnectionAge; PermitWithoutStream

Lộ trình tiếp theo

Chủ đề	Nội dung	Độ ưu tiên
gRPC-Gateway	Expose gRPC service như REST API cho browser	Cao
Health Check	grpc.health.v1 protocol — cần cho Kubernetes	Cao
Load Balancing	Client-side LB với DNS resolver, xDS protocol	Cao
Observability	OpenTelemetry tracing, Prometheus metrics, structured logging	Cao
Retry Policy	Service config JSON, backoff, hedging	Trung bình
Compression	gzip, snappy compression cho large payloads	Trung bình
Custom Resolver	Tích hợp với etcd, Consul service discovery	Nâng cao
gRPC Web	Dùng gRPC trực tiếp từ browser JavaScript	Nâng cao

Tài nguyên học tập

Docs chính thức

- grpc.io/docs/languages/go — Go specific gRPC documentation
- pkg.go.dev/google.golang.org/grpc — Go API Reference
- protobuf.dev — Protocol Buffers Language Guide

Repositories quan trọng

- github.com/grpc/grpc-go — Official Go gRPC library + examples
- github.com/grpc-ecosystem/go-grpc-middleware — Interceptor collection (logging, auth, retry, recovery)
- github.com/grpc-ecosystem/grpc-gateway — gRPC → REST transcoding
- github.com/fullstorydev/grpcurl — CLI tool để test gRPC

Sách

- "gRPC: Up and Running" — Kasun Indrasiri, Danesh Kuruppu (O'Reilly) — Bản tiếng Anh tốt nhất
- "Cloud Native Go" — Matthew A. Titmus — Chương về gRPC trong microservices

Công cụ Production

- buf.build — Modern Protobuf toolchain: lint, breaking change detection, BSR registry
- [Evans](https://github.com/GoogleCloudPlatform/evans) — Interactive gRPC client với REPL
- [grpcui](https://github.com/fullstorydev/grpcui) — Web UI để test gRPC service (như Swagger UI cho REST)



Lời kết

gRPC không khó — nó chỉ khác REST. Khi bạn đã làm quen với tư duy proto-first và 4 loại RPC, phần còn lại (interceptor, metadata, TLS) chỉ là thêm layers vào nền tảng đó. Hãy build một service thực tế — đó là cách học hiệu quả nhất!

Lỗi thường gặp — Tổng hợp

Lỗi	Nguyên nhân	Cách xử lý
transport: Error while dialing...	Server chưa chạy hoặc sai địa chỉ/port	Kiểm tra server đang lắng nghe. Dùng: <code>nc -zv localhost 50051</code>
code = Unimplemented	Method chưa được implement hoặc không đăng ký đúng service	Kiểm tra server đã implement và đăng ký đúng interface chưa
rpc error: code = DeadlineExceeded	Request vượt quá timeout	Tăng timeout hoặc tối ưu server. Kiểm tra network latency
all SubConns are in TransientFailure	Client không thể kết nối đến bất kỳ server nào	Server chưa chạy hoặc DNS resolution thất bại
connection reset by peer	Server đóng kết nối đột ngột (crash hoặc MaxConnectionAge)	Kiểm tra server log. Implement retry với backoff
EOF khi gọi Unary RPC	Server return nil trước khi gửi response — logic bug	Kiểm tra server handler: phải return (response, nil) — không return (nil, nil)
panic: interface conversion	Generated code và .proto không sync	Regenerate code sau mỗi lần thay đổi .proto
too many pings	Client ping quá nhanh, vi phạm EnforcementPolicy	Tăng <code>keepalive.ClientParameters.Time</code> hoặc tắt <code>PermitWithoutStream</code>

